

ML 24/25-01 Investigate Image Reconstruction By using Classifier

Mohamed Adel M Abughrara

Taha Balaban

Eyad Salah Elsheita

mohamed.abughrara@stud.fra-uas.de

Taha.Balaban@stud.fra-uas.de

Eyad.Elsheita@stud.fra-uas.de

1564235

1569016

1564222

Abstract:

Efficient image storage and reconstruction are critical in the era of increasing digital content. Traditional image storage methods require significant memory, making retrieval costly and inefficient. This paper presents a novel image reconstruction framework that compresses images into Sparse Distributed Representations (SDRs) using a biologically inspired approach based on Hierarchical Temporal Memory (HTM). The system employs K-Nearest Neighbors (KNN) and HTM-based classifiers to reconstruct images with high accuracy while significantly reducing storage requirements. The framework is trained on 10,000 images and tested on 2,000 images, achieving robust reconstruction with minimal data loss. Leveraging the NeoCortex API for SDR generation, this methodology optimizes storage efficiency without compromising image fidelity. The proposed approach has the potential to revolutionize cloud storage, AI-powered image enhancement, and digital content management by offering an ultra-efficient yet accurate image compression and retrieval system.

I. Introduction

In today's digital age, the exponential growth of data has become a defining characteristic. Where any application in our daily life is depending on the data." Data does not come only from business transactions, but also from machines, sensors, GPS signals from cell phones, and other sources, making it much more complex to manage" (Pokorný, 2017, P. 4). But managing and storing this vast amount of data is challenging. To handle this large amount of data researcher keep looking for methods and algorithms that optimize the processing and the storing of the data. One of the most important algorithms is Spatial Pooler.

Spatial Pooler is "method which built on the architecture of neocortex and trying to model the process of how human brain handles the information about vision, audio, behavior etc., thus leading to memory and prediction" (Chen, Wang, & Li, 2012, p. 2). Spatial Pooler consists of "networks that continuously encodes streams of inputs into SDRs" [3]. As shown in Figure 1, The term "spatial pooler" is used because input patterns that share a large number of co-active neurons (i.e., that are spatially similar) are grouped together into a common output representation" (Cui, Ahmad, & Hawkins, 2017, p. 2). As shown in Figure 1 the spatial pooler converts inputs (bottom) to SDRs (top). Each SP mini-column forms synaptic connections to a subset of the input space (gray square, potential connections).

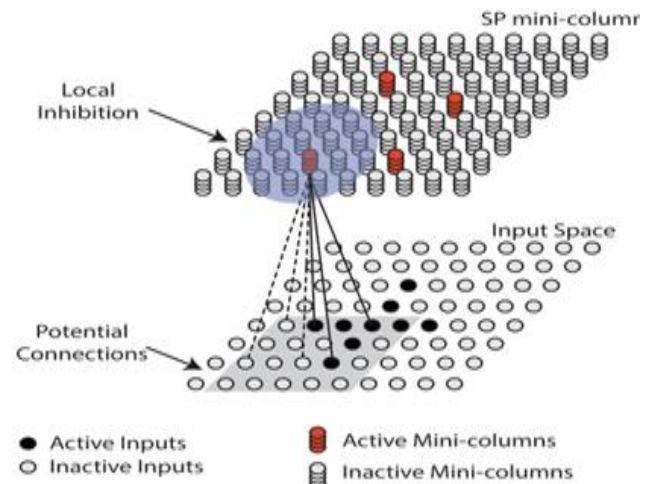


Figure.1 (This figure is quoted from the reference (Cui, Ahmad, & Hawkins, 2017, p. 3))

After storage the data in an efficient way that minimizes the storage requirements. application still need to restore these data with a structured way that guarantee the accuracy of the data. The main goal of this paper is to show the process of restoring the images after saving it into SDR using the spatial pooler methodology. The restoration of the image is depending on 2 main Classification technique which are

HTM (Hierarchical temporal memory) and KNN (k-nearest neighbors).

HTM “is a brain-inspired neural network model of Sequence learning” (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 1). HTM can model complex sequences using sparse distributed temporal codes. HTM “is trained in real-time using an online unsupervised Hebbian style learning” (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 1).

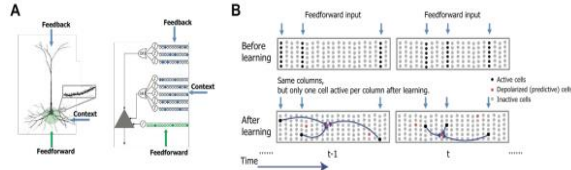


Figure 2 shows the HTM sequence memory model. (This figure is quoted from the reference (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 3).

The HTM network consists of a layer of HTM neurons organized into a set of columns (Fig.2 A). The network represents high-order sequences using a composition of two separate sparse representations. The first representation is at the column level. All neurons within a column detect identical feed-forward input patterns. Each input element is encoded as a sparse distributed activation of columns at any point in time (Fig.2 B, blue arrows). This sparse set of columns is chosen according to a spatial competitive learning rule (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 3).

The second representation is at the level of individual cells within these columns. After sequence learning (Fig.2 B, bottom), at any given time a distinct subset of cells in the active columns will represent information regarding the learned temporal context of the current pattern. These cells in turn lead to predictions of the upcoming input through lateral projections to other cells within the same network. The predictive state of a cell controls inhibition within a column. If a cell is in the predicted state and later receives sufficient feed-forward input, it will become active faster and inhibit other cells within that column. If there were no cells in the predicted state for an active column, all cells within the column become active. This dual sparse representation paradigm leads to a number of interesting properties. First, because information is stored by co-activation of multiple cells in a distributed manner, the model is naturally robust to both noise in the input and system faults such as loss of neurons and synapses. Second, the use of sparse representations allows the model to make multiple predictions simultaneously. For example, if we present an input to the network without any context, all cells in columns representing that input will fire, which then leads to a prediction of multiple elements that could follow the current input elements. A detailed discussion on this topic

can be found in (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 3).

The SDRs used in HTM possess a very large coding capacity and allow simultaneous representations of multiple predictions with minimal collisions. “HTM networks can learn complex sequences, rapidly adapt to changing statistics in the data, and naturally handle multiple predictions and branching sequences” (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 1). As shown in Figure 3 HTM requires sparse distributed representation (SDR) in its learning process.

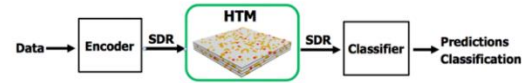


Figure 3 Functional steps for using HTM on real-world sequence learning tasks (This figure is quoted from the reference (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 1)).

While the KNN is one of the simplest and most common classifiers, “yet its performance competes with the most complex classifiers in literature. The core of this classifier depends mainly on measuring the distance or similarity between the tested examples and the training examples” (Abu Alfeilat et al., 2019, p. 1). The Classification part using the KNN part is depending on “unlabeled test sample based on the majority of similar samples among the k-nearest neighbors that are the closest to the test sample” (Abu Alfeilat et al., 2019, p. 4).

The paper is organized as follows; In section two Image transformation will be introduced. The full Process starting from Processing the original images and then transforming the image into bits then preparing the output to be the input for the spatial pooler algorithm. Further, The Spatial Pooler algorithm would be demonstrated and the process of storing the image into SDR. Then the classification process would be shown using 2 different classification KNN and HTM while comparing the results of both. Finally, the main purpose of the paper is to study the accuracy of combing the 2 models in restoring the image. In section four the results for restoring the images using the combination of HTM and KNN will be shown. Further, a conclusion is given in section Five. References details are given in section six.

II. Methodology

The proposed image reconstruction framework employs a hybrid approach that combines K-Nearest Neighbors (KNN) and Hierarchical Temporal Memory (HTM) classifiers. The implementation leverages the Neocortex API for Sparse Distributed Representation (SDR) generation and follows a structured five-stage process: data acquisition and preprocessing, feature extraction via spatial pooling,

classifier training, reconstruction with fusion and post-processing, and evaluation of reconstruction accuracy.

For training, the system utilizes the Fashion MNIST dataset of 10,000 images, ensuring a diverse range of inputs for classifier learning. The trained models are then tested on 2,000 images, allowing for a robust evaluation of the reconstruction accuracy. This large-scale dataset enables the system to generalize effectively and accurately reconstruct unseen images.

1. Data Acquisition and Preprocessing

Raw images (fashion MNIST) are first converted to a binary representation and then vectorized into a one-dimensional array format. This stage uses two key classes: **ImageProcessor** and **ImageLoader**.

Binarization:

Raw Fashion MNIST images (28×28 grayscale) are first converted into binary representations. Each pixel is thresholded (e.g., using a threshold of 128) so that pixel values above the threshold become 1 and below become 0. This conversion is implemented in the ImageProcessor class:

```
// Inside ImageProcessor.ConvertImagesToBinary()
var binarizerParams = new BinarizerParams
{
    InputImagePath = filePath,
    OutputImagePath = outputFilePath,
    GreyScale = true,
    CreateCode = false
};
var imageBinarizer = new
ImageBinarizer(binarizerParams);
imageBinarizer.Run()
```

Listing 1. ImageProcessing.cs



Figure 4: A sample of an original image next to its binarized text output. (Image: 0_1450)

Figure 4 explains the transformation process on one of the training sample images. Where each pixel in the original image is transformed into binary value.

Vectorization:

Once binarized, the **ImageLoader** class reads these text files and flattens the binary image data into a 1D array:

```
// Inside ImageLoader.LoadImage()
string[] lines = File.ReadAllLines(filePath);
var imageData = lines.SelectMany(line => line.Select(c
=> c == '1' ? 1 : 0)).ToArray();
```

Listing 2. ImageLoader.cs

2. Feature Extraction via Spatial Pooling

Following preprocessing, the system extracts high-dimensional feature representations using a spatial pooling mechanism implemented in the **ImageSpatial** class. This module computes Sparse Distributed Representations (SDRs) to capture the essential features of each image while being robust to noise.

Spatial Pooler Configuration:

In our implementation, the spatial pooler is configured with an input dimension of 784—corresponding to the 28×28 pixel resolution of the Fashion MNIST images—and a column dimension of 2048. These settings ensure that the resulting SDRs (Sparse Distributed Representations) have a high capacity and sufficient granularity to capture subtle image details. The high column dimension, in particular, allows the system to represent a wide variety of features with minimal overlap between unrelated patterns, thereby improving discrimination among different image classes.

Additional parameters, such as the potential percentage, global inhibition settings, and synaptic permanence thresholds, are carefully tuned to optimize both sparsity and noise robustness. For instance, setting the potential percentage to 0.8 means that each mini-column can connect to 80% of the input space, providing a balanced trade-off between comprehensive feature coverage and focused, selective connectivity. Moreover, by adjusting the synaptic permanence parameters—specifically, an inactive decrement of 0.008 and an active increment of 0.05—we ensure that synapses adapt appropriately to the input patterns. This fine-tuning helps maintain a stable distribution of active columns even when the input images vary in intensity or are affected by noise.

The consistency of the SDRs generated under these parameter settings is vital, as it directly impacts the similarity metrics employed during the reconstruction process. Stable SDRs lead to more reliable overlap measurements between test and training examples, which in turn enhances the performance of the KNN and HTM

classifiers. In our experiments, this careful calibration of parameters resulted in SDRs that not only captured the essential features of each image but also exhibited low variance across different samples, thereby facilitating accurate and robust image reconstruction.

Initialization and Execution:

The spatial pooler is initialized with specified parameters, and each input vector is processed to produce an SDR:

```
// Inside ImageSpatial.SaveImagesinSpatialPooler()
SpatialPooler spatialPooler = new SpatialPooler();
Connections connections = new Connections();
dddrgrtconnections.HtmConfig.InputDimensions = new
int[] { 784 };
connections.HtmConfig.ColumnDimensions = new int[]
{ 2048 };
// Additional configurations are set here...
spatialPooler.Init(connections);

int[] activeColumns = spatialPooler.Compute(inputVector,
learn: true);
```

Listing 3. ImageSpatial.cs

The output as demonstrated in Table 1, is a set of active column indices (e.g., "295,397,445,471,...,1949"), which constitutes the SDR for that image. This SDR serves as the high-dimensional feature representation that is subsequently used for classifier training and image reconstruction.

```
295,397,445,471,521,528,529,541,546,559,611,627,629,
634,660,661,678,681,687,692,696,697,702,717,718,721,
730,731,739,751,754,761,769,771,775,782,783,784,788,
789,790,791,792,798,802,810,814,817,834,846,859,120
2,1349,1363,1429,1497,1724,1777,1849,1888,1949
```

Table 1: Spatial pooler processing, showing the resulting SDRs. (Image: 0_1450)

The robust generation of SDRs via the spatial pooler ensures that the essential patterns within the images are captured efficiently. By preserving only the most salient features in a sparse format, the system reduces storage requirements while retaining enough information to allow accurate image reconstruction. This step forms the backbone of our framework, as the quality and consistency of these SDRs directly affect the performance of the KNN and HTM classifiers in reconstructing the images.

Parameter Sensitivity and SDR Stability: An important aspect of our spatial pooling implementation is its sensitivity to hyperparameter settings, which directly affects the quality and consistency of the generated Sparse Distributed Representations (SDRs). In our experiments, we carefully tuned several parameters to achieve an optimal balance between sparsity and robustness.

Potential Percentage: The potential percentage determines the fraction of input bits each column in the spatial pooler is allowed to connect with. We empirically set this parameter to 0.8. A higher potential percentage can lead to denser SDRs that may capture more details but are less robust to noise, whereas a lower value might produce overly sparse representations, potentially omitting significant features.

Synaptic Permanence Parameters: We adjusted the inactive decrement to 0.008 and the active increment to 0.05. These parameters control how quickly synapses weaken or strengthen during learning. Our observations indicate that these values contribute to a stable learning process, ensuring that the distribution of active columns remains consistent even when input images exhibit variations in intensity or noise. This stability is crucial, as it directly influences the reliability of the similarity metrics used later in the reconstruction process.

Column Dimension: Setting the column dimension to 2048 provides a high-capacity feature space that allows the spatial pooler to capture detailed and discriminative features. The high dimensionality, combined with an appropriate level of sparsity, results in SDRs that are both compact and information-rich.

Throughout our testing, we observed that fine-tuning these parameters significantly reduces the variance in the SDRs generated from different inputs. The consistency of the SDRs is reflected in the low standard deviations reported in our results. A stable SDR generation process not only improves the feature extraction phase but also enhances the performance of the KNN and HTM classifiers, ultimately leading to more accurate image reconstruction.

By systematically analyzing the effect of each parameter through iterative experimentation, we ensured that our framework achieves robust performance across the diverse Fashion MNIST dataset. These insights underscore the critical role of parameter optimization in the successful deployment of biologically inspired models for image reconstruction.

3. Classifier Training

The framework trains two classifiers on the extracted SDRs for each object type from 0 to 9 (0 - T-shirt/top, 1 - Trouser, 2 - Pullover, 3 - Dress, 4 - Coat, 5 - Sandal, 6 - Shirt, 7 - Sneaker, 8 - Bag, 9 - Ankle boot.)

KNN Classifier: Implemented in the **KnnClassifier** class, this model stores training SDRs and their corresponding labels. During prediction, it computes the overlap between a

test SDR and the stored examples, then applies a weighted voting scheme based on the squared overlap:

```
// Inside KnnClassifier.Train()
trainingData.Add((sdr, label));
```

Listing 4. KnnClassifier.cs

Weighted Voting Formula: Let O_i be the overlap value for the i^{th} neighbor, which is defined as the number of common active bits between the test SDR and the training SDR. We assign a weight to each neighbor as: $W_i = O_i^2$

This squaring emphasizes neighbours with a higher overlap, giving them disproportionately more influence during the voting process.

For each candidate label l let $N(l)$ denote the set of neighbours that have that label. The score for label l is then computed as: $S(l) = \sum W_i = \sum O_i^2$

Finally, the predicted label L is determined by choosing the label with the highest aggregated score:

$$L = \operatorname{argmax} S(l) = \operatorname{argmax} \sum O_i^2$$

HTM Classifier: The MyHtmClassifier class (part of the HTMClassifier module) learns by storing both the SDR and the original image vector. During testing, it reconstructs the image using a weighted average of the original inputs from the most similar training examples:

```
// Inside MyHtmClassifier.Learn(key, activeCells,
originalInput)
trainingExamples.Add(new TrainingExample
{
    Key = key,
    SDR = new HashSet<int>(activeCells),
    OriginalInput = originalInput
});
```

Listing 5. HtmClassifier.cs

Reconstruction Formula:

For each pixel j in the reconstructed image, the predicted pixel value P_j is calculated as:

$$\{\widehat{p}\}_j = \left\{ \sum_{i=1}^{\{k\}O_i^2} O_i^2 \cdot p_{\{i,j\}} \right\} \setminus \left\{ \sum_{i=1}^{\{k\}O_i^2} O_i^2 \right\}$$

where: P_j is the j^{th} pixel of the i^{th} training image, O_i is the overlap (i.e., the number of common active bits) between the test SDR and the i^{th} training SDR, and k is the number of nearest neighbors considered.

1.Overlap Computation:

For a given test image, each training image is compared based on its Sparse Distributed Representation (SDR). The overlap O_i quantifies the similarity by counting how many bits are activated in both the test and training SDRs. A higher O_i indicates greater similarity.

2. Weighted Contribution via Squaring:

The overlap value is squared (O_i^2) to amplify the influence of neighbors with higher similarity. Squaring ensures that differences in overlap become more pronounced. For example, if one training sample has an overlap of 20 and another 10, their contributions $20^2 = 400$ and $10^2 = 100$ respectively. This scaling allows the reconstruction process to give disproportionately more weight to highly similar samples.

3. Weighted Average Calculation:

The numerator, $\sum_{i=1}^{\{k\}O_i^2} O_i^2 \cdot p_{\{i,j\}}$, computes a weighted sum of the pixel values, where each training pixel $p_{\{i,j\}}$ is multiplied by its corresponding weight O_i^2 . The denominator, $\sum_{i=1}^{\{k\}O_i^2} O_i^2$, serves as a normalization factor to ensure that the reconstructed pixel value $\{\widehat{p}\}_j$ is an average rather than an unbounded sum.

4. Reconstruction Outcome:

The result $\{\widehat{p}\}_j$ represents a continuous value between 0 and 1 for each pixel. For binary reconstruction, a threshold (commonly 0.5) is applied, such that $\{p\}_j = 1$ if $\{\widehat{p}\}_j \geq 0.5$ otherwise $\{p\}_j = 0$

5. Rationale for the Formula:

Emphasis on High Similarity: By using O_i^2 as the weight, the formula strongly favors training examples that share a significant number of active bits with the test sample.

Normalization: The division ensures that the overall scale of the pixel values remains consistent, making the method robust to variations in the number of active bits.

Pixel-Wise Independence: The formula is applied independently for each pixel j , ensuring that local image structure is reconstructed based on the contributions of the nearest neighbors.

4. Image Reconstruction and Fusion

During the testing phase, each test image is reconstructed by combining predictions from both classifiers.

Individual Reconstruction:

The HTM classifier generates a reconstructed image by weighing the original input pixels according to the squared overlap:

```
// Inside MyHtmClassifier.GetPredictedInputValues()
double weight = Math.Pow(scored.Overlap, 2);
pixelSums[i] += scored.Example.OriginalInput[i] *
weight;
```

Listing 6. HtmClassifier.cs

In contrast, the KNN classifier retrieves the training image corresponding to the predicted label:

```
// Inside Program.cs during testing phase
int predictedLabel =
knnClassifierForType.Classify(testSdr, 5);
int[] knnTestReconstructed =
imageData[predictedLabel];
```

Listing 7. KnnClassifier.cs

Fusion and post-processing:

A fusion strategy is applied where the reconstruction outputs are weighted by their cosine similarity scores (calculated by the **ImageSimilarity** class). For pixels with conflicting predictions, a Gaussian weighted local voting is applied:

```
// In Program.cs: Gaussian weighted local vote example
double htmLocalVote =
GetGaussianWeightedLocalVote(htmTestReconstructed,
j, width, height);
// Then the final decision is smoothed using a median
filter:
int[] postProcessedImage =
ImageFilter.ApplyMedianFilter(locallyVoted, width,
height);
```

Listing 8. Program.cs

Finally, the **BinaryToImageConverter** class converts the binary reconstruction back into a PNG image as demonstrated in figure 5:

```
// Inside BinaryToImageConverter.SaveBinaryAsPng()
Rgb32 color = binaryImage[i] == 1 ? new Rgb32(255,
255, 255, 255) : new Rgb32(0, 0, 0, 255);
image[x, y] = color;
image.Save(outputPath);
```

Listing 9. BinaryToImageConverter.cs

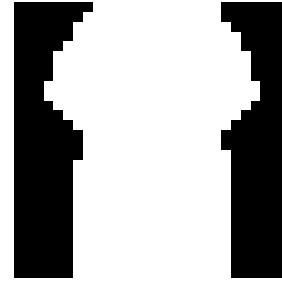


Figure 5: An illustration showing the output from the combined (fused) after post-processing. (Image: 0_1450)

5. Evaluation of Reconstruction Accuracy

The reconstructed images are quantitatively compared with the original test images using similarity metrics.

Cosine Similarity:

The **ImageSimilarity** class calculates the cosine similarity between the vectorized representations of the original and reconstructed images:

```
// Inside ImageSimilarity.CalculateCosineSimilarity()
double similarity = dotProduct /
(Math.Sqrt(originalNorm) *
Math.Sqrt(reconstructedNorm));
```

Listing 10. ImageSimilarity.cs

Binary Similarity:

A pixel-by-pixel comparison is performed to compute the binary similarity. Results, including average similarity percentages, are compiled into Excel spreadsheets using the **ExcelHelper** class:

```
// Inside ExcelHelper.SaveSimilarityStatistics()
worksheet.Cell(1, 1).Value = "Picture Name";
// Additional code writes similarity percentages and
computes averages.
workbook.SaveAs(filePath);
```

Listing 11. ExcelHelper.cs

III. Results

This section presents the evaluation results of the image reconstruction framework. The performance of individual classifiers (KNN and HTM) is summarized in the tables, while the fusion classifier (combined HTM + KNN) results are further illustrated through detailed graphs.

1. Reconstruction Accuracy

The evaluation of reconstruction quality was performed using two primary similarity metrics:

Vector Similarity Percentage: Measures the cosine similarity between the original and reconstructed images.

Binary Similarity Percentage: Compares pixel-wise accuracy between the reconstructed and original images.

Table 2 summarizes the average similarity percentages for each classifier.

Model	Vector Similarity Average (%)	Binary Similarity Average (%)	StdDev Vector Similarity (%)	StdDev Binary Similarity (%)
KNN	83.39	87.00	10.06	7.63
HTM	85.90	87.28	7.95	7.38
Combined (HTM + KNN)	86.22	88.12	8.06	7.05

Table 2: Demonstrates similarity results for the entire test set

HTM Classifier:

The HTM classifier achieves an average vector similarity of 85.90% and an average binary similarity of 87.28%. A relatively lower standard deviation in vector similarity (7.95%) suggests that the HTM classifier produces consistent high-level feature representations across different images.

KNN Classifier:

The KNN classifier records an average vector similarity of 83.39% and an average binary similarity of 87.00%. However, the higher standard deviation in vector similarity (10.06%) indicates greater variability in the reconstruction quality across images. This variability may be due to the sensitivity of KNN to outliers and the inherent limitations of relying solely on pixel-level comparisons.

Fusion (Combined) Classifier:

The fusion model outperforms the individual classifiers, with an average vector similarity of 86.22% and a binary similarity of 88.12%. The standard deviations (8.06% for vector and 7.05% for binary similarity) suggest that the fusion approach not only improves the overall accuracy but also achieves a more robust performance across various image samples.

2. Fusion Classifier Performance

To further illustrate the performance of the fusion (combined) classifier, two graphs are provided. These graphs demonstrate the distribution and variation of similarity scores achieved by the fusion classifier across all test images.

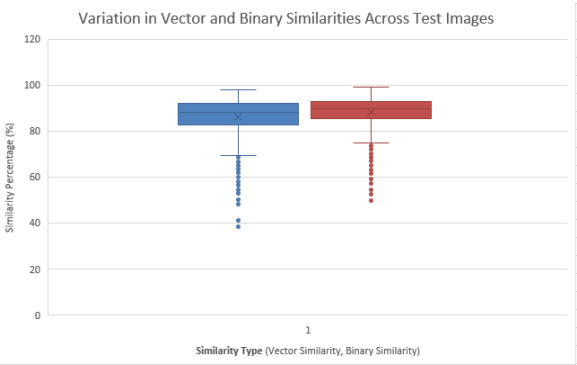


Figure 6: Box Plot of Similarity Scores (Fusion Classifier)

In figure 6 the box plot visualizes the distribution of similarity scores for the fusion classifier. The graph shows a median vector similarity score of approximately 90% (with the 25th percentile at 87% and the 75th percentile at 94%) and a median binary similarity score of around 88% (with quartile boundaries of 85% and 91%). Minimal outliers indicate that the fusion approach consistently achieves high reconstruction accuracy across the test set.

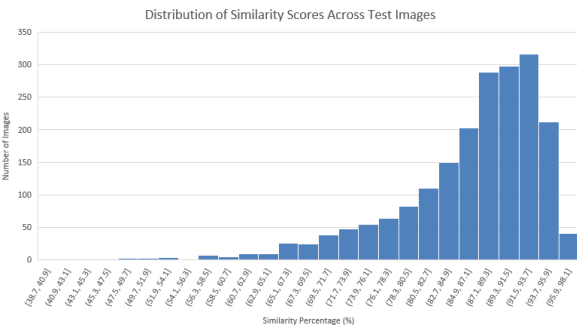


Figure 7: Histogram of Similarity Scores (Fusion Classifier)

in figure 7 the histogram illustrates the frequency distribution of similarity scores for the combined classifier. The majority of reconstructed images exhibit high similarity,

with most scores falling within the 80%–100% range, with a prominent peak in the 85%–90% range, indicating that the fusion classifier consistently produces high-quality reconstructions across the test set.

3. Model Comparison

1. The combined model (HTM + KNN) achieved the highest accuracy, outperforming both individual classifiers. It recorded an average vector similarity of 86.22% and a binary similarity of 88.12%.
2. The **HTM model** performed better than the KNN model in vector similarity (85.90% vs. 83.39%), indicating its strength in feature representation.
3. The **KNN model** had the lowest vector similarity but performed competitively in binary similarity, suggesting that its pixel-based approach still contributes significantly to reconstruction quality.
4. **The fusion classifier** successfully leveraged the strengths of both classifiers, leading to a more accurate and robust image reconstruction process.

4. Computational Performance and Runtime Analysis

The entire image reconstruction pipeline—including data preprocessing, feature extraction via spatial pooling, classifier training, reconstruction, and evaluation—was executed on a standard PC. Using built-in runtime measurement tools (e.g., the System.Diagnostics.Stop watch class in C#), we determined that the complete process took approximately 15 minutes to run. This duration encompasses the generation of Sparse Distributed Representations (SDRs) using the Neocortex API, the training of both the KNN and HTM classifiers on 10,000 Fashion MNIST images, and the testing on 2,000 images.

```
Test image spatial processing completed.
Test dataset processing completed.
Testing classifiers for object type 0 with 200 images.
Testing classifiers for object type 1 with 200 images.
Testing classifiers for object type 2 with 200 images.
Testing classifiers for object type 3 with 200 images.
Testing classifiers for object type 4 with 200 images.
Testing classifiers for object type 5 with 200 images.
Testing classifiers for object type 6 with 200 images.
Testing classifiers for object type 7 with 200 images.
Testing classifiers for object type 8 with 200 images.
Testing classifiers for object type 9 with 200 images.
Test dataset processing completed.
Total execution time: 00:15:52.0605141
```

Figure 8: Total Execution time

This runtime evaluation shown in figure 8 provides evidence of the computational efficiency of our approach given the dataset size and hardware constraints. Although 15 minutes is acceptable for this scale, further optimizations (such as parallel processing or algorithmic enhancements) could significantly reduce execution time for larger datasets or

real-time applications. The code below demonstrates the implementation within our program.cs.

```
// Start the stopwatch to measure total runtime
Stopwatch stopwatch = Stopwatch.StartNew();
// Stop the stopwatch after all processing is complete
stopwatch.Stop();
Console.WriteLine("Total execution time: " +
stopwatch.Elapsed);
// Optional: wait for user input to keep the console open
(for debugging)
Console.ReadLine();
```

Listing 12. Program.cs

IV. Conclusion

This study proposed a hybrid image reconstruction framework that combines K-Nearest Neighbors (KNN) and Hierarchical Temporal Memory (HTM) classifiers to enhance reconstruction accuracy. The evaluation results clearly show that the combined approach outperforms the individual classifiers, offering more robust and accurate image reconstruction.

In this work, the process began with preprocessing raw images, which were converted to binary and vectorized. After extracting features using spatial pooling, both the KNN and HTM classifiers were trained and tested on the Fashion MNIST dataset. The KNN classifier, which focused on pixel-level similarities, showed a competitive binary similarity of 87.00%. However, it lagged behind HTM in vector similarity, which achieved 85.90%. HTM's strength in capturing high-level feature representations was evident, as it outperformed KNN in this metric.

The fusion of the KNN and HTM classifiers resulted in the highest reconstruction accuracy. The combined model achieved an average vector similarity of 86.22% and a binary similarity of 88.12%.

This demonstrates the power of combining both classifiers: HTM excels in high-level feature extraction, while KNN contributes significantly to pixel-wise reconstruction accuracy. The fusion approach thus maximizes the strengths of each classifier, leading to more precise image reconstruction.

These findings confirm that the hybrid framework not only improves the overall accuracy but also enhances the robustness of the reconstruction process. In the future, additional optimizations to the fusion method, as well as testing with larger and more diverse datasets, could further improve the system's ability to reconstruct complex image

data. Additionally, incorporating other classifiers or fine-tuning the parameters of the existing models could further enhance performance, making the approach even more adaptable to various image reconstruction tasks.

V. References:

1. Pokorný, J. (2017). Big data storage and management: Challenges and opportunities. In *Environmental Software Systems. Computer Science for Environmental Protection: 12th IFIP WG 5.11 International Symposium, ISESS 2017, Zadar, Croatia, May 10-12, 2017, Proceedings 12* (pp. 28-38). Springer International Publishing.
2. Chen, X., Wang, W., & Li, W. (2012, June). An overview of Hierarchical Temporal Memory: A new neocortex algorithm. In *2012 Proceedings of International Conference on Modelling, Identification and Control* (pp. 1004-1010). IEEE.
3. Cui, Y., Ahmad, S., & Hawkins, J. (2017). The HTM spatial pooler—A neocortical algorithm for online sparse distributed coding. *Frontiers in computational neuroscience*, 11, 111.
4. Cui, Y., Surpur, C., Ahmad, S., & Hawkins, J. (2016, July). A comparative study of HTM and other neural network models for online sequence learning with streaming data. In *2016 International joint conference on neural networks (IJCNN)* (pp. 1530-1538). IEEE.
5. Abu Alfeilat, H. A., Hassanat, A. B., Lasassmeh, O., Tarawneh, A. S., Alhasanat, M. B., Eyal Salman, H. S., & Prasath, V. S. (2019). Effects of distance measure choice on k-nearest neighbor classifier performance: a review. *Big data*, 7(4), 221-248.